

Preguntas frecuentes

Práctica 3 – Sistemas operativos

[\(Go to English version tab\)](#)

Errores comunes detectados en la Práctica 2 (Tomarlo en cuenta para la Práctica 3):

* Nombre incorrecto del archivo ZIP. * NIA(s) incorrectos o mal indicados. * Formato incorrecto en autores.txt. * No ejecutar el script checker_sso_p2.py antes de entregar. * No probar el programa en Guernika. * Entregar zips duplicados; varios envíos por equipo. * No colocar suficientes comentarios en sus códigos.	* En la memoria, aseguraos de incluir todos los apartados solicitados exactamente como se especifica. Además, por favor, preparad una <u>memoria clara y de buena calidad</u>. * No utilizar el Makefile para compilar (make). * No utilizar el formato de salida solicitado. * No utilizar el código de error correcto como -1 * Ser detectado varias veces en el sistema de plagio.
--	--

En caso de dudas no resueltas en este documento, contacta con el profesor de prácticas de tu grupo reducido:

Número de grupo reducido	Nombre	Correo
80	Javier Rodríguez	jlines@inf.uc3m.es
81	Juan Carlos Cano	jucanor@pa.uc3m.es
84, 85 y 801	Genaro Sánchez	gesanche@pa.uc3m.es
87 y 141	Catherine Torres	cattorre@inf.uc3m.es
88 y 89	Paula Sánchez	paulasan@inf.uc3m.es

Preguntas generales

1. ¿Qué valor representa la mayor prioridad en las entradas del estacionamiento?

Se utilizan los valores de 0 a 2 siendo:

0 - Autos ecológicos o de cero emisiones (prioridad máxima).

1 - Residentes (prioridad media).

2 - Público general o visitantes (prioridad mínima).

2. En caso de que no se ponga la cabecera (o se ponga mal), Number Plate, Gate, Priority, deberían dar error o seguir la ejecución sin tenerlo en cuenta.

Para esta práctica no hace falta comprobar la cabecera, puedes asumir que estará ahí siempre.

- 3. ¿Habría que hacer una batería de pruebas para cada función, queue.c, etc, es decir, pruebas unitarias, o con hacer pruebas globales sobre el funcionamiento correcto del programa principal valdría al igual que con las prácticas anteriores?**

No exigimos hacer pruebas unitarias. Sin embargo, si las haces te aseguras de que tu programa funciona bien.

- 4. ¿Se deben controlar las matrículas duplicadas?**

No es necesario comprobar que las matrículas sean únicas. En caso de que haya matrículas duplicadas, se procesarán igualmente.

- 5. ¿Se tienen que ordenar los valores de entrada por prioridad antes de comenzar el acceso al parking?**

No. Los valores de entrada se procesan en el orden en que se leen. La prioridad solo se tiene en cuenta en el momento de gestionar el acceso al parking.

- 6. El enumerado de las puertas de salida, ¿Sería 0 y 1 o 1 y 2? En el caso de las puertas de entrada comienzan en el cero, pero en la representación gráfica que nos han proporcionado del parking se indica q_put(exit_q1) y q_put(exit_q2) para las de salida.**

Utiliza **0 y 1** para enumerar las puertas de salida; en la figura exit_q1 y exit_q2 son los nombres de las variables que hemos dado a las colas circulares.

- 7. Cuando todas las líneas del CSV son inválidas o solo tenga la cabecera y no haya ningún coche, ¿debemos devolver -1 y que pare el programa, o que ejecute los hilos pero al ver que no hay coches finalice el programa?**

Si todas las líneas son inválidas aplica la misma regla: "En caso de que una línea no sea válida, se deberá ignorar y continuar con la siguiente.". El comportamiento en el escenario que planteas sería la ejecución de los hilos, mostrar el mensaje "Waiting" (con el formato que hemos pedido) y luego finalizarían con el mensaje "Ending" (con el formato que hemos pedido).